LangChain Document Loaders

A Complete Learning Guide

PyPDFLoader · WebBaseLoader · CSVLoader · UnstructuredMarkdownLoader

Created: June 24, 2026 03:26

Author: Sourav Bhattacharya

Vector Databases | RAG Pipelines | Document Ingestion | Metadata Enrichment | LangChain | Python

Index

1. The LangChain Document Object

Foundation of every document pipeline

Every LangChain loader — regardless of whether the source is a PDF, a website, a CSV, or a Markdown file — returns a Python list of **Document** objects. Understanding this two-field dataclass is essential before using any loader.

What is a Document?

```
from langchain_core.documents import Document

# A Document has exactly two fields
doc = Document(
    page_content = 'The actual text extracted from the source.',
    metadata     = {
        'source': 'path/to/file.pdf',   # always present
        'page'  : 0,                    # loader-specific
        # ... additional loader-specific keys
    }
)

# Access fields
print(doc.page_content[:100])          # first 100 chars of text
print(doc.metadata)                    # {'source': ..., 'page': 0}
```

Document Pipeline — Overview Diagram

Figure 1 — End-to-end RAG pipeline. The Document object carries both text and metadata through the entire pipeline.

Key Insight: Metadata is stored alongside each vector in the vector database. Missing metadata at load time means you cannot filter on it during retrieval. Enrich early.

2. PyPDFLoader

langchain_community.document_loaders

PyPDFLoader uses the **pypdf** library to parse PDF files. It creates **one Document per page**, making page-level chunking natural.

2.1 Installation

```
pip install langchain-community pypdf
```

2.2 How PyPDFLoader works (diagram)

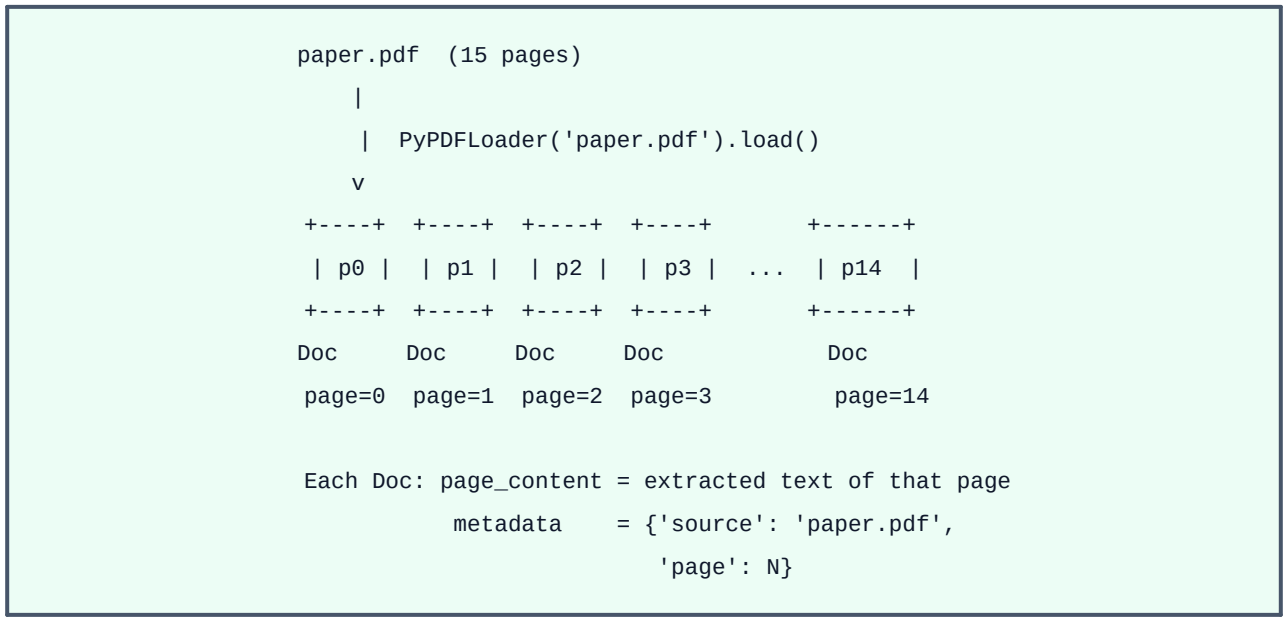

Figure 2 — PyPDFLoader splits a PDF into one Document per page.

2.3 Code Example: load PDF, print page count & page 2 content

```
from langchain_community.document_loaders import PyPDFLoader

# ■■ Step 1: Load the PDF ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
loader = PyPDFLoader('attention_is_all_you_need.pdf')
pages  = loader.load()

# ■■ Step 2: Print number of pages ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print(f'Total pages: {len(pages)}')

# ■■ Step 3: First 500 characters of page 2 (index 1) ■■■■■■■■■■
page2 = pages[1]
print('--- Page 2 content (first 500 chars) ---')
print(page2.page_content[:500])
```

```
# ■■ Step 4: Inspect the Document metadata ■■■■■■■■■■■■■■■■■■■■■■
print('--- Metadata ---')
print(page2.metadata)
```

```
Total pages: 15
--- Page 2 content (first 500 chars) ---
2 Background
The goal of reducing sequential computation also forms the foundation of the
Extended Neural GPU [16], ByteNet [18] and ConvS2S [9], all of which use
convolutional neural networks as basic building block, computing hidden
representations in parallel for all input and output positions...
--- Metadata ---
{'source': 'attention_is_all_you_need.pdf', 'page': 1}
```

2.4 Lazy loading for large PDFs

```
# Use lazy_load() to avoid loading all pages into RAM at once
for page in loader.lazy_load():
    process(page)    # chunk -> embed -> upsert one page at a time

# Load an entire folder of PDFs
from langchain_community.document_loaders import PyPDFDirectoryLoader
all_pages = PyPDFDirectoryLoader('./papers/').load()
```

2.5 Metadata: Captured vs. Missing

✓ Captured automatically	■ Missing — add manually
✓ source (file path string)	■ title / author (PDF properties)
✓ page (0-based integer)	■ creation_date / modification_date
	■ page_count (total pages in document)
	■ file_size (bytes on disk)
	■ load_timestamp (ingestion time)
	■ doc_id (SHA-256 hash for deduplication)

2.6 How to add the missing metadata

```
import pypdf, hashlib, datetime

path   = 'attention_is_all_you_need.pdf'
reader = pypdf.PdfReader(path)
pdf_m  = reader.metadata            # /Title, /Author, /CreationDate

pages  = PyPDFLoader(path).load()
for doc in pages:
    doc.metadata['title']          = pdf_m.get('/Title', 'Unknown')
```

```
    doc.metadata['author']         = pdf_m.get('/Author', 'Unknown')
    doc.metadata['creation_date']  = str(pdf_m.get('/CreationDate', ''))
    doc.metadata['page_count']     = len(reader.pages)
    doc.metadata['load_timestamp'] = datetime.datetime.utcnow().isoformat()
    doc.metadata['doc_id']         = hashlib.sha256(
        doc.page_content.encode()).hexdigest()[:16]
```

3. WebBaseLoader

langchain_community.document_loaders

WebBaseLoader fetches HTML with **requests** and parses it with **BeautifulSoup4**. It returns **one Document per URL**. Best for static pages; for JavaScript-rendered sites use Playwright or FireCrawl.

3.1 Installation

```
pip install langchain-community beautifulsoup4
```

3.2 How WebBaseLoader works (diagram)

Figure 3 — WebBaseLoader: HTTP fetch → HTML parse → one Document per URL.

3.3 Code Example: load Wikipedia article

```
import bs4
from langchain_community.document_loaders import WebBaseLoader

url = 'https://en.wikipedia.org/wiki/Vector_database'

loader = WebBaseLoader(
    web_paths=[url],
    # Restrict BeautifulSoup to the article body only
```

```
    bs_kwargs={
        'parse_only': bs4.SoupStrainer(
            class_=('mw-parser-output',)
        )
    },
)
docs = loader.load()

# ■■ Inspect the Document object ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print(f'Documents loaded : {len(docs)}')          # 1
print(f'Content length   : {len(docs[0].page_content)} chars')

print('\npage_content (first 300 chars):')
print(docs[0].page_content[:300])

print('\nmetadata:')
print(docs[0].metadata)
```

```
Documents loaded : 1
Content length   : 18340 chars

page_content (first 300 chars):
A vector database, vector store or vector search engine is a database that
can store vectors (fixed-length lists of numbers) along with other data items.
Vector databases typically implement one or more Approximate Nearest Neighbor
(ANN) algorithms...

metadata:
{'source': 'https://en.wikipedia.org/wiki/Vector_database',
 'title': 'Vector database - Wikipedia',
 'description': 'From Wikipedia, the free encyclopedia',
 'language': 'en'}
```

> **Tip:** Always use bs4.SoupStrainer to target only the article body. Without it, nav bars, footers, and cookie banners clutter page_content and waste embedding tokens.

3.4 Metadata: Captured vs. Missing

✓ Captured automatically	■ Missing — add manually
✓ source (full URL)	■ fetch_timestamp
✓ title (HTML tag)	■ http_status_code (200 / 404 etc.)
✓ description (meta description)	■ last_modified (HTTP header)
✓ language (HTML lang attr)	■ domain (from URL)
	■ word_count
	■ section / heading breadcrumb

3.5 How to add the missing metadata

```
import requests, datetime
from urllib.parse import urlparse

for doc in docs:
    url  = doc.metadata['source']
    resp = requests.head(url, allow_redirects=True, timeout=10)
    doc.metadata['fetch_timestamp']  = datetime.datetime.utcnow().isoformat()
    doc.metadata['http_status_code'] = resp.status_code
    doc.metadata['last_modified']    = resp.headers.get('Last-Modified', '')
    doc.metadata['domain']           = urlparse(url).netloc
    doc.metadata['word_count']       = len(doc.page_content.split())
```

4. CSVLoader

langchain_community.document_loaders.csv_loader

CSVLoader reads a CSV file and produces **one Document per row**. Column names become part of the text by default, or you can promote specific columns to metadata using **metadata_columns=**.

4.1 Installation

```
pip install langchain-community
```

4.2 How CSVLoader works (diagram)

```
                products.csv
                 product_id | name          | category    | price
                 P001       | Laptop Pro 15 | Electronics | 1299.99
                 P002       | Office Chair  | Furniture   | 349.99
                 P003       | Coffee Maker  | Appliances  | 89.99
                     |
                      |  CSVLoader('products.csv').load()
                     v
           [ Doc(page_content='product_id: P001\nname: Laptop Pro 15\n...',
                      metadata={'source':'products.csv', 'row':0}),
             Doc(page_content='product_id: P002\nname: Office Chair\n...',
                      metadata={'source':'products.csv', 'row':1}),
                Doc(..., metadata={..., 'row':2}) ]
```

Figure 4 — CSVLoader creates one Document per row; column names are included in page_content.

4.3 Sample CSV — products.csv

```
product_id,name,category,price,rating,in_stock
P001,Laptop Pro 15,Electronics,1299.99,4.8,True
P002,Office Chair,Furniture,349.99,4.2,True
P003,Coffee Maker,Appliances,89.99,4.6,False
P004,Wireless Mouse,Electronics,29.99,4.5,True
```

4.4 Code Example: basic load

```
from langchain_community.document_loaders.csv_loader import CSVLoader

loader = CSVLoader(file_path='products.csv')
docs   = loader.load()

print(f'Documents loaded: {len(docs)}')    # 4 (one per data row)
```

```
print(docs[0].page_content)
# product_id: P001
# name: Laptop Pro 15
# category: Electronics
# price: 1299.99
# rating: 4.8
# in_stock: True

print(docs[0].metadata)
# {'source': 'products.csv', 'row': 0}
```

4.5 Promote columns to metadata

```
loader = CSVLoader(
    file_path='products.csv',
    source_column='product_id',         # use product_id as source
    metadata_columns=['product_id','category','price','rating','in_stock'],
    content_columns=['name'],            # only 'name' in page_content
)
docs = loader.load()

print(docs[0].page_content)    # name: Laptop Pro 15
print(docs[0].metadata)
# {'source': 'P001', 'row': 0, 'product_id': 'P001',
#  'category': 'Electronics', 'price': '1299.99',
#  'rating': '4.8', 'in_stock': 'True'}
```

4.6 Metadata: Captured vs. Missing

✓ Captured automatically	■ Missing — add manually
✓ source (file path or source_column value)	■ file_last_modified (file mtime)
✓ row (0-based integer)	■ total_rows (row count)
✓ Any column via metadata_columns= parameter	■ column_names (header schema)
	■ load_timestamp
	■ null_count (data quality per row)

4.7 How to add the missing metadata

```
import os, datetime, csv

path  = 'products.csv'
mtime = datetime.datetime.fromtimestamp(os.path.getmtime(path)).isoformat()

with open(path) as f:
    rows = list(csv.DictReader(f))
```

```
docs = CSVLoader(file_path=path).load()
for i, doc in enumerate(docs):
    doc.metadata['file_last_modified'] = mtime
    doc.metadata['total_rows']         = len(rows)
    doc.metadata['load_timestamp']     = datetime.datetime.utcnow().isoformat()
    doc.metadata['null_count']         = sum(
        1 for v in rows[i].values() if v == '')
```

5. UnstructuredMarkdownLoader

langchain_community.document_loaders

UnstructuredMarkdownLoader uses the **unstructured** library to parse Markdown. It has two modes: **single** (whole file = one Document) and **elements** (each heading / paragraph / list = its own Document with rich metadata).

5.1 Installation

```
pip install langchain-community 'unstructured[md]'
```

5.2 single vs. elements mode (diagram)

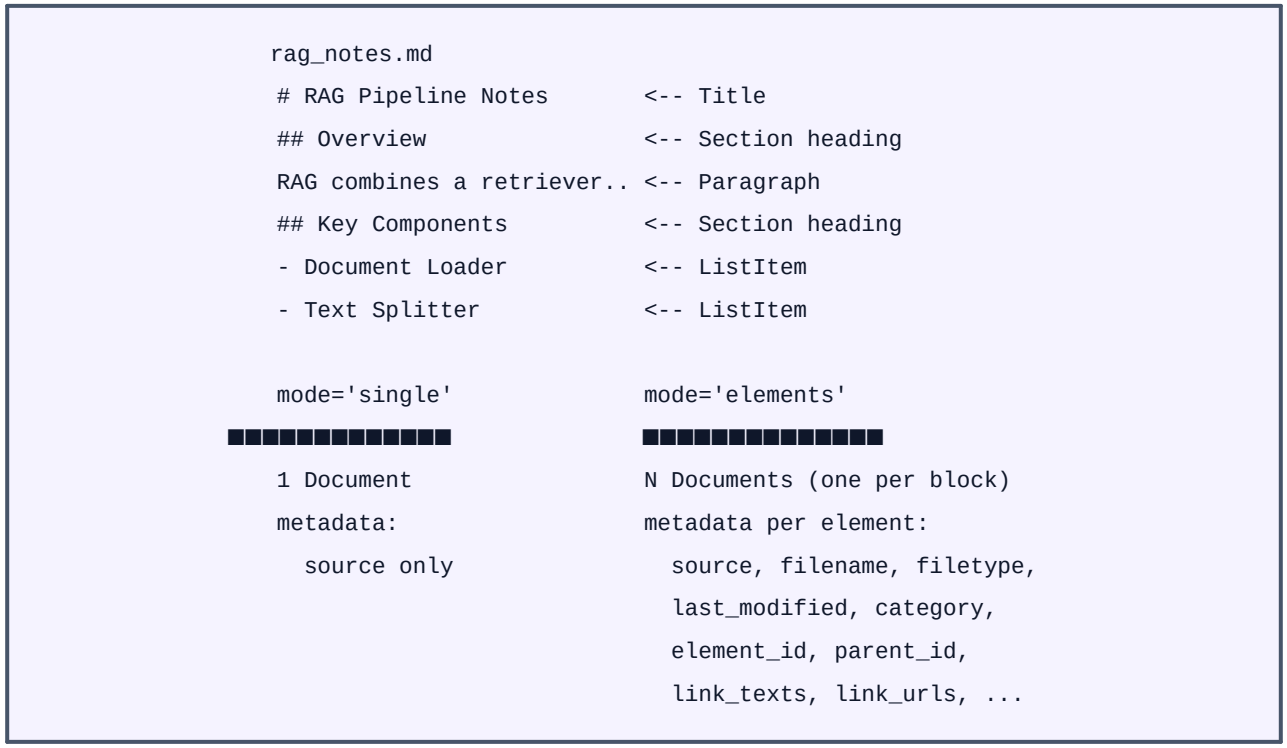

Figure 5 — single mode collapses the file to one Document; elements mode produces one Document per structural block with rich metadata.

5.3 Sample Markdown — rag_notes.md

```
# RAG Pipeline Notes

## Overview
RAG (Retrieval-Augmented Generation) combines a retriever
with a generative model to answer questions grounded in docs.

## Key Components
```

```
- **Document Loader** -- ingests raw sources
- **Text Splitter**   -- breaks docs into chunks
- **Embeddings**      -- converts text to vectors
- **Vector Store**    -- stores and retrieves vectors

## Useful Links
See the [LangChain docs](https://python.langchain.com).
```

5.4 Code Example — single mode

```
from langchain_community.document_loaders import UnstructuredMarkdownLoader

loader = UnstructuredMarkdownLoader('rag_notes.md', mode='single')
docs   = loader.load()

print(f'Documents: {len(docs)}')          # 1
print(docs[0].page_content[:200])
print(docs[0].metadata)
# {'source': 'rag_notes.md'}
```

5.5 Code Example — elements mode (recommended for RAG)

```
loader = UnstructuredMarkdownLoader('rag_notes.md', mode='elements')
docs   = loader.load()

print(f'Elements: {len(docs)}')           # e.g. 8-12

# First element is always the Title
print(docs[0].page_content)               # RAG Pipeline Notes
print(docs[0].metadata)
```

```
Elements: 9
RAG Pipeline Notes
{
  'source':         'rag_notes.md',
  'filename':       'rag_notes.md',
  'file_directory': '.',
  'filetype':       'text/markdown',
  'last_modified':  '2026-06-23T10:30:00',
  'languages':      ['eng'],
  'category':       'Title',
  'element_id':     'b2f4a9c1...',
  'parent_id':      None,
  'link_texts':     [],
  'link_urls':      []
}
```

5.6 Metadata: Captured vs. Missing

✓ Captured automatically	■ Missing — add manually
✓ source, filename, file_directory	■ heading_path (H1 > H2 > H3 breadcrumb)

✓ filetype (text/markdown)	■ heading_level (1, 2, 3)
✓ last_modified (file mtime)	■ word_count / char_count
✓ languages (['eng'])	■ frontmatter fields (YAML title, author, tags)
✓ category (Title / NarrativeText / ListItem)	■ load_timestamp
✓ element_id (content hash)	■ doc_id (deduplication hash)
✓ parent_id (links block to heading)	
✓ link_texts / link_urls	

5.7 How to add the missing metadata

```
import yaml, datetime, hashlib

def get_frontmatter(path):
    with open(path) as f:
        content = f.read()
    if content.startswith('---'):
        end = content.index('---', 3)
        return yaml.safe_load(content[3:end])
    return {}

fm   = get_frontmatter('rag_notes.md')
now  = datetime.datetime.utcnow().isoformat()

for doc in docs:
    doc.metadata['load_timestamp'] = now
    doc.metadata['doc_id']         = hashlib.sha256(
        doc.page_content.encode()).hexdigest()[:16]
    doc.metadata['word_count']     = len(doc.page_content.split())
    doc.metadata.update(fm)         # merge YAML front-matter
```

6. Metadata Comparison — All Four Loaders

The table below shows which metadata keys each loader captures automatically, and how many Documents it generates per source file.

Key	PyPDFLoader	WebBaseLoader	CSVLoader	UnstructuredMD
`source`	✓ file path	✓ URL	✓ file/col	✓ file path
`page / row`	✓ page int	—	✓ row int	—
`title`	—	✓	—	—
`description`	—	✓ meta tag	—	—
`language`	—	✓ html lang	—	✓ detected
`filename`	—	—	—	✓
`filetype`	—	—	—	✓ MIME
`last_modified`	—	—	—	✓ mtime
`category`	—	—	—	✓ element type
`element_id`	—	—	—	✓
`link_urls`	—	—	—	✓ (elements)
`author`	Manual	Manual	Manual	Manual
`load_ts`	Manual	Manual	Manual	Manual
`doc_id`	Manual	Manual	Manual	Manual

Documents produced per source file

Loader	Source	Docs produced	Granularity
PyPDFLoader	1 PDF	1 per page	Page
WebBaseLoader	1 URL	1 per URL	Full page
CSVLoader	1 CSV	1 per row	Row
UnstructuredMarkdown	1 .md	1 (single) / N (elems)	File / Block

7. Universal Enrichment Function

The enrichment pattern below works with any loader. Run it immediately after **loader.load()**, before passing documents to the text splitter or embeddings.

7.1 Enrichment flow (diagram)

Figure 6 — Enrich metadata right after loading, before splitting or embedding.

7.2 Python implementation

```
import hashlib, datetime
from langchain_core.documents import Document

def enrich(
    docs: list[Document],
    source_type: str,
    extra: dict = None
) -> list[Document]:
    '''
    Add universal metadata to any list of Documents.

    Args:
        docs        - list returned by any loader.load()
        source_type - 'pdf' | 'web' | 'csv' | 'markdown'
        extra       - optional additional key-value pairs
    Returns:
```

```
        same list, mutated in place
    '''
    now = datetime.datetime.utcnow().isoformat()
    for doc in docs:
        doc.metadata['doc_id']         = hashlib.sha256(
            doc.page_content.encode()).hexdigest()[:16]
        doc.metadata['load_timestamp'] = now
        doc.metadata['source_type']    = source_type
        doc.metadata['char_count']     = len(doc.page_content)
        doc.metadata['word_count']     = len(doc.page_content.split())
        if extra:
            doc.metadata.update(extra)
    return docs

# ■■ Usage examples ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
pdf_docs = enrich(
    PyPDFLoader('paper.pdf').load(),
    source_type='pdf',
    extra={'project': 'rag-v2', 'team': 'ml-platform'}
)

web_docs = enrich(
    WebBaseLoader(['https://en.wikipedia.org/wiki/Vector_database']).load(),
    source_type='web',
    extra={'crawl_depth': 1, 'freshness': 'weekly'}
)

csv_docs = enrich(
    CSVLoader('products.csv').load(),
    source_type='csv',
    extra={'schema_version': '1.0'}
)
```

Vector DB Tip: In Chroma use add_documents(docs). In Pinecone, metadata= becomes the vector's metadata dict. In Weaviate, it becomes properties. All three support equality and range filters on numeric metadata fields like word_count or page.

8. Quick-Reference Cheat Sheet

Install all four loaders

```
pip install langchain-community pypdf beautifulsoup4 'unstructured[md]'
```

All imports

```
from langchain_community.document_loaders import (
    PyPDFLoader, WebBaseLoader, UnstructuredMarkdownLoader
)
from langchain_community.document_loaders.csv_loader import CSVLoader
```

One-liner per loader

```
pages = PyPDFLoader('paper.pdf').load()                          # 1 per page
web   = WebBaseLoader(['https://...']).load()                     # 1 per URL
rows  = CSVLoader('data.csv').load()                              # 1 per row
md    = UnstructuredMarkdownLoader('notes.md',mode='elements').load()  # 1 per block
```

Universal inspect pattern

```
docs = loader.load()
print(f'Docs loaded: {len(docs)}')
print(f'Keys: {list(docs[0].metadata.keys())}')
print(docs[0].page_content[:300])
print(docs[0].metadata)
```

Key questions for any new loader

- How many Documents does one source produce? (page / row / element / whole-file)
- Which metadata keys are auto-captured? Print doc.metadata right after load().
- Which columns / fields belong in page_content vs. metadata for your query patterns?
- Is lazy_load() available? Use it for large files to avoid OOM errors.
- Does the loader strip important formatting (tables, footnotes)? Consider alternatives.

9. References

All content in this guide is drawn from the official documentation sources listed below. Accessed June 2026.

[1] **LangChain — Document Loaders overview**
https://python.langchain.com/docs/concepts/document_loaders/

[2] **LangChain — PyPDFLoader API reference**
https://python.langchain.com/api_reference/community/document_loaders/langchain_community.document_loaders.pdf.PyPDFLoader.html

[3] **LangChain — WebBaseLoader API reference**
https://python.langchain.com/api_reference/community/document_loaders/langchain_community.document_loaders.web_base.WebBaseLoader.html

[4] **LangChain — CSVLoader API reference**
https://python.langchain.com/api_reference/community/document_loaders/langchain_community.document_loaders.csv_loader.CSVLoader.html

[5] **LangChain — UnstructuredMarkdownLoader**
https://python.langchain.com/api_reference/community/document_loaders/langchain_community.document_loaders.markdown.UnstructuredMarkdownLoader.html

[6] **Unstructured library — Elements and metadata**
https://docs.unstructured.io/open-source/core-functionality/partitioning

[7] **pypdf — PDF reading library**
https://pypdf.readthedocs.io/en/stable/

[8] **BeautifulSoup4 documentation**
https://www.crummy.com/software/BeautifulSoup/bs4/doc/